Codex:

Architecture of the project ‘portfolio management system’:

This is a **Java 21 Spring MVC web API** packaged as a WAR for a Jakarta Servlet container such as Tomcat. It uses Spring’s Java configuration directly; the project does not use Spring Boot.

```
Browser or API client
        | JSON over HTTP
        ▼
Servlet container
  ├─ Spring Security filter chain
  └─ Spring MVC controllers
        |
        ▼
Application services
        |
        ▼
Repositories (JdbcTemplate)
        |
        ▼
MySQL database
```

The app starts through [WebAppInitializer.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/config/WebAppInitializer.java). It creates a root Spring context for application services, persistence, database, and security configuration, plus a web context for controllers and MVC settings. [RootConfig.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/config/RootConfig.java) scans the application and persistence packages and imports the database and security setup.

The main layers are:

- **API:** Controllers accept HTTP requests and return JSON. For example, [PortfolioController.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/api/PortfolioController.java) handles portfolio routes.
- **Application:** Services validate inputs and apply business rules, such as [PortfolioService.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/application/PortfolioService.java) and [HoldingService.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/application/HoldingService.java).
- **Persistence:** Repositories run database queries using Spring `JdbcTemplate`.
- **Database and configuration:** [DatabaseConfig.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/config/DatabaseConfig.java) configures the MySQL connection pool and transaction manager. The schema is applied manually from `src/main/resources/db/schema.sql`.

Typical request workflow

For an authenticated portfolio request, the flow is:

1. The client sends a request to an `/api/...` endpoint.
2. Spring Security checks the session and, for `POST`, `PUT`, or `DELETE`, the CSRF token.
3. The controller gets the signed-in user and calls the relevant service.
4. The service validates the request and asks a repository to read or update MySQL.
5. The controller returns a JSON response with an HTTP status.

For example, creating a portfolio goes from `POST /api/portfolios` to `PortfolioController`, then `PortfolioService`, then `PortfolioRepository`. The user ID comes from the authenticated session, so portfolio operations are scoped to that user.

Main user flows

- **Register and sign in:** Registration hashes passwords with Argon2id. Login checks credentials and uses MySQL-backed throttling. Successful login creates a server-side session and changes its ID. The client first gets a CSRF token from `GET /api/auth/csrf`; it sends that token and the matching cookie with state-changing requests.
- **Manage portfolios:** A signed-in user can create, list, open, and delete their own portfolios. A portfolio must have no holdings before it can be deleted.
- **Manage holdings:** A user can add, list, update, and delete current positions. The service checks ownership, validates units and prices using decimal values, and prevents duplicate investment names within a portfolio.

The project's current scope is an API for accounts, portfolios, and current holdings. The README and requirements mention a broader portfolio-management vision, but live prices, transaction history, recommendations, and a user interface are outside the implemented MVP scope. See [README.md](C:/Users/Admin/portfolio-management-system/README.md) and [REQUIREMENTS.md](C:/Users/Admin/portfolio-management-system/REQUIREMENTS.md).

A manageable MVP could include:

- **User profile:** goals, time horizon, and risk tolerance
- **Investment options:** shares, mutual funds, fixed deposits, and other categories
- **Risk and return comparisons:** show assumptions and data sources
- **Portfolio tracker:** let users record holdings and view allocation
- **Educational guidance:** explain why an option may or may not fit a user's profile

For the Spring backend, you could use Spring Boot with Spring Web, Spring Data JPA, and a database such as PostgreSQL. Core models might include `User`, `InvestmentOption`, `Portfolio`, `Holding`, and `RiskProfile`.

I can help you turn this into a requirements document, design the database and API, or build it in your existing project.

We'll use the **Spring Framework without Spring Boot**. That means setting up the web application, dependency injection, database access, and server configuration explicitly.

I can adapt the project in your workspace to plain Spring. What should we build first: the project structure, the database model, or a basic user and portfolio API?

Project Structure:

```
src/
├── main/
│   ├── java/com/portfolio/
│   │   ├── config/
│   │   │   ├── WebConfig.java
│   │   │   ├── RootConfig.java
│   │   │   └── DatabaseConfig.java
│   │   ├── user/
│   │   │   ├── UserController.java
│   │   │   ├── UserService.java
│   │   │   ├── UserRepository.java
│   │   │   └── User.java
│   │   ├── investment/
│   │   │   ├── InvestmentController.java
│   │   │   ├── InvestmentService.java
│   │   │   ├── InvestmentRepository.java
│   │   │   └── Investment.java
│   │   ├── portfolio/
│   │   │   ├── PortfolioController.java
│   │   │   ├── PortfolioService.java
│   │   │   ├── PortfolioRepository.java
│   │   │   ├── Portfolio.java
│   │   │   └── Holding.java
│   │   ├── recommendation/
│   │   │   ├── RecommendationController.java
│   │   │   └── RecommendationService.java
│   │   └── common/
│   │       └── exception/
```

```
│   ├── resources/
│   │   └── database.properties
│   └── webapp/
│       └── WEB-INF/
└── test/
    ├── java/com/portfolio/
    └── resources/
```

`WebInitializer` registers Spring MVC with the servlet container. The config classes set up component scanning, database access, and transactions. Controllers handle HTTP requests, services contain application logic, repositories access the database, and model classes represent stored data.

Your current project already uses a Maven WAR layout and has Spring MVC configuration, but its `pom.xml` also includes Spring Boot libraries. To keep this project strictly on Spring Framework, those Boot dependencies will need to be removed when we adapt the existing setup.

Suggested 10-day plan

Day	What we'll build	What you'll learn
1	Explore the existing project and how it starts	Maven layout, Spring configuration, and the request flow
2	Add the portfolio package structure	How to organize code by feature
3	Define the first domain classes: `User`, `Investment`, `Portfolio`, and `Holding`	Classes, fields, relationships, and basic domain modeling
4	Add database mappings and configuration	JPA entities and how Spring connects to a database
5	Create repositories for storing and retrieving data	The repository pattern and basic database operations
6	Add service classes for user and portfolio operations	Where business logic belongs and dependency injection
7	Add controllers and a few basic endpoints	HTTP requests, JSON, and Spring MVC
8	Add validation and error handling	Handling invalid input and returning useful errors
9	Add a simple, explainable investment comparison feature	Applying business rules while making uncertainty and risk clear
10	Review the application and document how to run it	Following the full flow and explaining the project

We can adjust the pace as you go—one "day" can take more than one session. For the investment feature, we'll present comparisons as educational guidance and avoid promising returns.

Day 1 will start by tracing a simple request through your existing code. We haven't changed any files yet.

We can include that as a core feature: when a portfolio holding is down, the system can show other investment options for the user to compare.

We should design it to explain **possible alternatives and their risks**, rather than promise that another option will recover the loss or earn good returns. For example, it could compare historical performance, risk level, fees, and suggested time horizon, while noting that past performance does not guarantee future returns.

That fits into **Day 9** of our plan. We can first build the portfolio tracking and comparison pieces, then add simple, transparent rules for suggesting alternatives based on the user's goals and risk tolerance.

[ApiInfoController.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/common/ApiInfoController.java) handles two simple HTTP GET requests:

- `GET /api` returns the API name, a `"ready"` status, and the health endpoint path.
- `GET /api/health` returns `{"status":"UP"}` as a basic health check.

It's a small placeholder to confirm the Spring MVC app is responding. It doesn't check databases or other services. `ApiInfoController.java` is optional. We added it as a simple way to confirm the API responds; Spring MVC projects don't require one. Keep it if you want those two endpoints, or remove it when you start building the actual features.

[WebAppInitializer.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/config/WebAppInitializer.java) tells a Jakarta Servlet container how to start the Spring MVC application. It registers Spring's `DispatcherServlet` at `/` and points it to the configuration in `WebConfig.java`.

It's required for this project's current setup because we're deploying a WAR without a `web.xml`. You could use `web.xml` to configure the servlet instead, but you need one of those approaches so the container knows how to start the app.

The `DispatcherServlet` is the core "Front Controller" of the Spring MVC framework. It acts as the central traffic cop for your web application, intercepting all incoming HTTP requests, coordinating with underlying components, and returning the final response back to the client.

[WebConfig.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/config/WebConfig.java) configures Spring MVC for the web application:

- `@EnableWebMvc` enables Spring MVC features such as request mapping and controller handling.
- `@ComponentScan("com.portfolio")` tells Spring to find components such as `ApiInfoController`.

`WebAppInitializer.java` loads this configuration when the servlet container starts the app.

These files are part of the starter structure we set up, not a mandatory checklist for every Java project.

For this project's current approach—Spring MVC deployed as a WAR without Spring Boot—the key pieces are:

- `WebAppInitializer.java` so the servlet container can start and configure the app.
- `WebConfig.java` to enable MVC and register Spring components.

`ApiInfoController.java` is optional; it only provides sample API and health endpoints. As the project grows, you can add Java files for its actual features.

The next step is to define the **first feature the system should support** before adding more code. That will shape the data model and API.

A good first milestone is: **a user can create a portfolio, add investments, and view its holdings**. We can write down the requirements for that flow, then design the database tables and API endpoints.

Database design:

A good starting schema would include:

- `users` — account details and password hash
- `portfolios` — owner, name, and timestamps
- `holdings` — portfolio, investment name and normalized name, optional ticker, units, average purchase price, and timestamps
- `login_attempts` — data for the reviewed throttling rules

The main remaining schema detail is how to represent MySQL login throttling windows and expiry. We can make that concrete while designing the tables.

The normalized name in ‘holdings’ table, gives the database a consistent value to use when checking for duplicates.

For example, `Reliance Industries`, `reliance industries`, and `RELIANCE INDUSTRIES` would all be stored as the same normalized value. A unique constraint on `(portfolio_id, normalized_investment_name)` can then prevent the same investment from being added twice to one portfolio.

The original name is still kept for display. Since tickers are optional, the name is the duplicate identity in the current requirements.

`RootConfig.java` is a Spring configuration class that brings together the application’s component scanning and other configuration classes.

- `package com.portfolio.config;`
  Places this class in the `com.portfolio.config` package.
- The three `import` lines make Spring’s `@ComponentScan`, `@Configuration`, and `@Import` annotations available to this file.
- `@Configuration`
  Marks `RootConfig` as a class that provides Spring configuration.
- `@ComponentScan(basePackages = {"com.portfolio.application", "com.portfolio.persistence"})`
  Tells Spring to look in those two packages for components, such as classes annotated with `@Component`, `@Service`, or `@Repository`, and register them as beans.
- `@Import({DatabaseConfig.class, SecurityConfig.class})`
  Includes the configuration from `DatabaseConfig` and `SecurityConfig` in the Spring context. This lets `RootConfig` serve as a central entry point for those configurations too.
- `public class RootConfig {}`
  Declares the configuration class. Its body is empty because the annotations provide its configuration.

When Spring loads `RootConfig`, it scans the specified packages and also loads the database and security configurations.

`RootConfig.java` is a Spring configuration class that gathers the application's setup in one place.

```
package com.portfolio.config;
```

This declares the package the class belongs to. It also means `RootConfig` can refer to other classes in `com.portfolio.config`—such as `DatabaseConfig` and `SecurityConfig`—without writing their full package names.

```
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;
```

These imports make Spring's configuration annotations available to this file.

```
@Configuration
```

This tells Spring that `RootConfig` is a source of configuration. Spring can read it when building the application context.

```
@ComponentScan(basePackages = {
    "com.portfolio.application",
    "com.portfolio.persistence"
})
```

This tells Spring to search those packages and their subpackages for component classes, such as classes annotated with `@Component`, `@Service`, or `@Repository`, and register them as beans.

```
@Import({DatabaseConfig.class, SecurityConfig.class})
```

This includes the configuration defined by `DatabaseConfig` and `SecurityConfig` in the same Spring application context. Those classes can provide additional beans and settings.

```
public class RootConfig {
}
```

This declares the `RootConfig` class. Its body is empty because the configuration is supplied by the annotations and the imported classes, rather than by methods in this class.

In short, when Spring loads `RootConfig`, it scans the application and persistence packages and also loads the database and security configuration.

Annotation	Description
`@Component`	The basic marker for any Spring-managed Java class.

`@Service`	Specialized version of `@Component` used to mark business logic layers.
`@Repository`	Specialized version of `@Component` for DAO/Data Access layers; automatically handles database exception translations.
`@Controller`	Marks a class as a Spring MVC controller that serves web views or pages.
`@Autowired`	Tells Spring to automatically resolve and inject dependent beans into fields, constructors, or setters.
`@Qualifier`	Used alongside `@Autowired` to specify exactly which bean to inject when multiple implementations of an interface exist.
`@Primary`	Gives preference to a specific bean when multiple candidates exist for a dependency.
`@Lazy`	Delays bean initialization until the moment it is actually requested rather than at application startup.
`@Scope`	Configures the bean's lifecycle (e.g., `singleton`, `prototype`, `request`, `session`).
@Bean	If you need to use a class from an external library (like Gson, Jackson, RestTemplate, or an Amazon S3 client), you cannot open their `.class` files to add a `@Component` annotation. Instead, you write a method that instantiates it, and mark it with `@Bean`.

`SecurityConfig.java` sets up authentication, sessions, CSRF protection, and access rules for the API. I'll go through it in the order Spring uses its declarations.

1. Package and imports

```
package com.portfolio.config;
```

This places the class in the `com.portfolio.config` package.

The `import` lines bring in the Spring Security classes used below. For example, `HttpSecurity` is used to define security rules, and `Argon2PasswordEncoder` provides password hashing.

2. Mark the class as security configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

- `@Configuration` tells Spring this class provides configuration.
- `@EnableWebSecurity` turns on Spring Security's web security support.
- `public class SecurityConfig` declares the configuration class.

3. Set up password hashing

```
@Bean
public PasswordEncoder passwordEncoder() {
    return Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8();
}
```

A **password encoder** hashes passwords before they are stored and checks a submitted password against its stored hash during login. Argon2 is the hashing algorithm selected here.

`@Bean` tells Spring to create and manage the returned object so other parts of the application can use it.

4. Configure how login credentials are checked

```
@Bean
public AuthenticationManager authenticationManager(
        UserAccountService userAccountService,
        PasswordEncoder passwordEncoder) {
    DaoAuthenticationProvider provider =
            new DaoAuthenticationProvider(userAccountService);
    provider.setPasswordEncoder(passwordEncoder);
    return new ProviderManager(provider);
}
```

- `UserAccountService` looks up an account by email.
- `DaoAuthenticationProvider` uses that service and the password encoder to check credentials.
- `ProviderManager` delegates authentication to that provider.
- The returned `AuthenticationManager` is used by the login code.

In this project, the login endpoint calls the authentication manager when a user submits their credentials.

5. Store the signed-in user in the session

```
@Bean
public SecurityContextRepository securityContextRepository() {
    return new HttpSessionSecurityContextRepository();
}
```

After a successful login, Spring stores the security context—which includes the authenticated user—in the server-side HTTP session. Later requests can use that session to recognize the user.

6. Change the session ID after login

```
@Bean
public SessionAuthenticationStrategy sessionAuthenticationStrategy() {
    return new ChangeSessionIdAuthenticationStrategy();
}
```

This changes the session ID when authentication succeeds. It helps prevent someone from reusing a session ID that was set before the user signed in.

7. Configure the CSRF token cookie

```
@Bean
public CsrfTokenRepository csrfTokenRepository() {
```

```
    CookieCsrfTokenRepository repository =
            CookieCsrfTokenRepository.withHttpOnlyFalse();
    repository.setCookiePath("/");
    repository.setCookieCustomizer(cookie -> cookie
            .sameSite("Lax")
            .secure("production".equalsIgnoreCase(System.getenv("APP_ENV"))));
    return repository;
}
```

CSRF protection helps prevent a malicious website from using a signed-in browser's session to make unwanted changes.

This repository stores the CSRF token in a cookie:

- `withHttpOnlyFalse()` lets browser-based code read the CSRF cookie and send its value in a request header.
- `setCookiePath("/")` makes the cookie available across the application.
- `sameSite("Lax")` restricts when browsers send the cookie with cross-site requests.
- `secure(...)` marks the cookie as HTTPS-only when `APP_ENV` is set to `production`.

The client first requests a token from `GET /api/auth/csrf`, then sends it with state-changing requests such as `POST`, `PUT`, or `DELETE`.

8. Define the security filter chain

```
@Bean
public SecurityFilterChain securityFilterChain(...) throws Exception {
    http
```

The **filter chain** is Spring Security's set of checks applied to incoming web requests. The rest of this method configures those checks.

Use the session to store security information

```
.securityContext(context ->
        context.securityContextRepository(securityContextRepository))
```

This tells Spring which repository to use for saving and loading the signed-in user's security context.

Create sessions only when needed and protect session IDs

```
.sessionManagement(session -> session
        .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
        .sessionFixation(fixation -> fixation.changeSessionId()))
```

- `IF_REQUIRED` means Spring creates a session when the application needs one, such as after login.
- `changeSessionId()` changes the session ID after authentication.

Enable CSRF protection

```
.csrf(csrf -> csrf.csrfTokenRepository(csrfTokenRepository))
```

This enables CSRF checks and tells Spring where to get the expected token.

Choose which routes require login

```
.authorizeHttpRequests(authorize -> authorize
        .requestMatchers(HttpMethod.GET,
                "/api", "/api/health", "/api/auth/csrf").permitAll()
        .requestMatchers(HttpMethod.POST,
                "/api/auth/register", "/api/auth/login").permitAll()
        .anyRequest().authenticated())
```

These rules allow anyone to:

- Check the API information and health endpoints.
- Get a CSRF token.
- Register or log in.

Every other request requires an authenticated user. For example, portfolio and holding operations require login.

Turn off saved-request behavior

```
.requestCache(cache -> cache.disable())
```

Spring Security often saves a page a user tried to open before being redirected to log in. This API disables that behavior because it returns API responses rather than navigating users between web pages.

Return JSON errors for access failures

```
.exceptionHandling(exceptions -> exceptions
        .authenticationEntryPoint((request, response, exception) -> {
            response.setStatus(401);
            response.setContentType("application/json");
            response.getWriter().write(
                    "{\"error\":\"authentication_required\"}");
        })
        .accessDeniedHandler((request, response, exception) -> {
            response.setStatus(403);
            response.setContentType("application/json");
            response.getWriter().write(
                    "{\"error\":\"access_denied\"}");
        }))
```

- **401 Unauthorized** is returned when the request needs a signed-in user but there isn't one.
- **403 Forbidden** is returned when the request is denied, such as when a CSRF check fails.

Configure logout

```
.logout(logout -> logout
        .logoutUrl("/api/auth/logout")
        .logoutSuccessHandler(
                (request, response, authentication) ->
                        response.setStatus(204)));
```

A `POST /api/auth/logout` request logs the user out. Spring's logout support clears the authentication and invalidates the session. On success, the API returns **204 No Content**, meaning the logout worked and there is no response body.

Finally:

```
return http.build();
```

This builds the configured filter chain so Spring Security can apply it to incoming requests.

`AuthController` handles authentication-related HTTP requests under the `/api/auth` URL prefix. It delegates account creation and credential checks to application services, while Spring Security manages the logged-in session.

1. Controller setup

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {
```

`@RestController` makes the methods HTTP endpoints and serializes their return values as JSON. `@RequestMapping` sets the shared URL prefix.

The constructor receives five dependencies:

- `UserAccountService accounts` creates user accounts.
- `LoginService loginService` checks credentials and reports whether login succeeded or was rate-limited.
- `SessionAuthenticationStrategy` applies session-related security steps after login.
- `SecurityContextRepository` saves the authenticated user's security context so it can be restored on later requests.
- `CsrfTokenRepository` manages the CSRF token used to protect state-changing requests.

Spring creates and supplies these dependencies.

2. Fetch a CSRF token

```
@GetMapping("/csrf")
public Map<String, String> csrf(@RequestAttribute("_csrf") CsrfToken csrfToken)
```

This handles `GET /api/auth/csrf`. Spring Security places the current CSRF token on the request as `_csrf`; the `@RequestAttribute` parameter retrieves it.

The method returns JSON containing the header name the client should use and the token value, for example:

```
{"headerName":"X-CSRF-TOKEN","token":"..."}
```

A client typically fetches this before sending a request such as login or registration that needs CSRF protection.

3. Register an account

```
@PostMapping("/register")
public ResponseEntity<?> register(@RequestBody RegisterRequest request)
```

This handles `POST /api/auth/register`. Spring reads the request's JSON body into a `RegisterRequest`, which contains `email` and `password`.

If the request object is missing, the method throws an `ApiException` with a `400 Bad Request` status. Otherwise:

1. It passes the email and password to `accounts.register(...)`.
2. It normalizes the email for the response.
3. It returns `201 Created` with the new user's ID and email.

The password is not included in the response. Validation and password storage details belong to `UserAccountService`.

4. Log in

```
@PostMapping("/login")
public ResponseEntity<?> login(...)
```

This handles `POST /api/auth/login`. It receives the JSON request plus the servlet request and response, which are needed for session and security operations.

First, it asks `loginService.authenticate(...)` to check the supplied email and password. It also passes the client's remote address, likely for login-rate tracking. The `request == null ? null : ...` checks let the method pass null credentials if the body is absent.

The result has three possible outcomes:

- **Rate limited:** returns `429 Too Many Requests` with a generic retry-later message.
- **Invalid credentials:** returns `401 Unauthorized` with a generic message.
- **Successful login:** continues with Spring Security setup.

On success, the method:

1. Gets the authenticated `Authentication` object.
2. Runs the session authentication strategy, which may apply session protections such as session-fixation handling.
3. Creates a fresh `SecurityContext` and puts the authentication in it.
4. Sets that context for the current request.
5. Saves it through the repository so future requests can recognize the logged-in user.
6. Clears the saved CSRF token so Spring Security can issue a fresh one after authentication.
7. Returns the user ID and email.

The method declares `throws Exception` because the session strategy or related servlet operations may throw checked exceptions.

5. Get the current user

```
@GetMapping("/me")
public Map<String, Object> me(...)
```

This handles `GET /api/auth/me`. Spring Security supplies the current authenticated principal as a `UserPrincipal`.

If there is no principal, the method throws an `AuthenticationServiceException`. Otherwise, it returns the user ID and username (the email) as JSON. Usually, the security configuration determines whether an unauthenticated request is rejected before it reaches this method.

6. Request body types

```
public record RegisterRequest(String email, String password) {}
public record LoginRequest(String email, String password) {}
```

These Java records are compact data classes Spring can populate from the JSON bodies for registration and login.

In short: registration delegates account creation to `UserAccountService`; login delegates credential checking to `LoginService` and then saves the successful authentication in the session; `/me` reports the current authenticated user; and `/csrf` gives the client a CSRF token.

`HoldingController` exposes HTTP endpoints for viewing and managing holdings. It receives the current user from Spring Security and passes that user's ID to `HoldingService`, which handles the actual operations.

1. Imports

The imports bring in:

- `List` and `Holding` for returning holdings.
- `HoldingService` for the business operations.
- `UserPrincipal` for identifying the logged-in user.
- Spring MVC annotations for routes, path variables, and JSON request bodies.
- `ResponseEntity` and `HttpStatus` for setting HTTP response statuses.

2. Controller declaration and service

```
@RestController
public class HoldingController {
    private final HoldingService holdings;
```

`@RestController` tells Spring that this class handles web requests and that its return values should be serialized into HTTP responses, usually as JSON.

`holdings` is the service the controller uses to list, add, update, and delete holdings.

```
public HoldingController(HoldingService holdings) {
    this.holdings = holdings;
}
```

Spring supplies a `HoldingService` when it creates the controller. This constructor-based dependency injection makes the service available to each endpoint.

3. List holdings in a portfolio

```
@GetMapping("/api/portfolios/{portfolioId}/holdings")
public List<Holding> list(...)
```

This handles a `GET` request such as:

```
/api/portfolios/12/holdings
```

- `@AuthenticationPrincipal` supplies the logged-in user as a `UserPrincipal`.
- `@PathVariable` reads `12` from the URL as `portfolioId`.
- The controller calls `holdings.list(user.userId(), portfolioId)`.
- The resulting list is returned as JSON.

Passing the user ID lets the service apply user-specific access checks or filtering.

4. Add a holding

```
@PostMapping("/api/portfolios/{portfolioId}/holdings")
public ResponseEntity<Holding> add(...)
```

This handles a `POST` request to the portfolio's holdings URL. The request body is converted from JSON into a `HoldingService.HoldingInput`.

The controller passes the user ID, portfolio ID, and input to `holdings.add(...)`. It wraps the returned holding in a response with status **201 Created**, indicating that a resource was created.

5. Update a holding

```
@PutMapping("/api/holdings/{holdingId}")
public Holding update(...)
```

This handles a `PUT` request such as:

```
/api/holdings/45
```

Spring reads `45` as the `holdingId` and converts the JSON body into `HoldingInput`. The controller calls `holdings.update(...)` with the user ID, holding ID, and new input. The updated holding is returned as JSON.

6. Delete a holding

```
@DeleteMapping("/api/holdings/{holdingId}")
public ResponseEntity<Void> delete(...)
```

This handles a `DELETE` request for a specific holding.

The controller asks the service to delete it, passing the current user ID and holding ID. If that call succeeds, it returns **204 No Content**, which means the deletion succeeded and there is no response body.

Overall flow

For each route, Spring matches the URL and HTTP method, fills in the user and request parameters, then calls the controller method. The controller delegates the work to `HoldingService`; the service's return value becomes the HTTP response.

From the Java files in this project, a useful learning order is:

1. **Controllers** — how HTTP requests enter the application. We've covered `AuthController` and `HoldingController`; next could be `PortfolioController`.
2. **Application services and models** — where the application's operations and data are defined, such as `PortfolioService` and `Portfolio`.
3. **Repositories** — how those services read and write data.
4. **Configuration** — how Spring, the database, and security are connected.
5. **Shared error handling and supporting classes** — how errors and other common concerns are handled.

`PortfolioController` defines HTTP endpoints for listing, creating, viewing, and deleting portfolios. It receives requests, gets the identity of the signed-in user, and delegates the actual work to `PortfolioService`.

1. Package and imports

```
package com.portfolio.api;
```

This places the class in the `api` part of the project, where the HTTP controllers live.

The imports bring in the types used by the controller:

- `Portfolio` and `PortfolioDetails` are data objects returned by its endpoints.
- `PortfolioService` performs the portfolio operations.

- `UserPrincipal` represents the signed-in user.
- Spring's annotations map HTTP requests to Java methods.
- `ResponseEntity` lets a method set the HTTP response status as well as its body.

2. Tell Spring this is a controller

```
@RestController
@RequestMapping("/api/portfolios")
public class PortfolioController {
```

`@RestController` tells Spring to treat this class as a web controller and convert returned Java objects into HTTP responses, usually JSON.

`@RequestMapping("/api/portfolios")` sets the common beginning of every URL in this controller. For example, `@GetMapping("/{portfolioId}")` below handles a URL like `/api/portfolios/7`.

3. Connect the controller to the service

```
private final PortfolioService portfolios;

public PortfolioController(PortfolioService portfolios) {
    this.portfolios = portfolios;
}
```

The controller needs `PortfolioService` to carry out operations. Spring supplies it when it creates the controller; this is called **dependency injection**.

The field is `final`, so it must be set by the constructor and cannot later be replaced.

4. List the signed-in user's portfolios

```
@GetMapping
public List<Portfolio> list(@AuthenticationPrincipal UserPrincipal user) {
    return portfolios.list(user.userId());
}
```

`@GetMapping` maps this method to `GET /api/portfolios`.

Spring supplies the signed-in user as `user`. The method passes that user's ID to `portfolios.list(...)` and returns the resulting list as JSON.

5. Create a portfolio

```
@PostMapping
public ResponseEntity<Portfolio> create(...)
```

`@PostMapping` maps this method to `POST /api/portfolios`.

The request JSON is converted into a `CreatePortfolioRequest`, which contains the portfolio's name. The method then calls:

```
portfolios.create(user.userId(), request == null ? null : request.name())
```

This passes the signed-in user's ID and the requested name to the service. The null check means that if the request body is absent, the method passes `null` as the name.

Finally, it returns the created portfolio with HTTP status **201 Created**.

6. Get one portfolio's details

```
@GetMapping("/{portfolioId}")
public PortfolioDetails get(...)
```

This maps to `GET /api/portfolios/{portfolioId}`, such as `GET /api/portfolios/7`.

Spring reads the number in the URL into `portfolioId`. The controller passes both the signed-in user's ID and the portfolio ID to `portfolios.get(...)`, then returns the details.

Passing the user ID lets the service apply its ownership or access rules.

7. Delete a portfolio

```
@DeleteMapping("/{portfolioId}")
public ResponseEntity<Void> delete(...)
```

This maps to `DELETE /api/portfolios/{portfolioId}`.

The controller passes the user ID and portfolio ID to `portfolios.delete(...)`. If that call succeeds, it returns **204 No Content**, meaning the deletion completed and the response has no body.

`public ResponseEntity<Void>` describes what this method returns:

- **`public`** means Spring and other code can call the method.
- **`ResponseEntity`** lets the method choose the HTTP response status and headers.
- **`Void`** means the response has no body.

Here, the method deletes the portfolio, then returns:

```
return ResponseEntity.noContent().build();
```

That sends HTTP **204 No Content**: the deletion succeeded, and there's no data to return. `Void` makes that clear in the method's return type.

It could return just `void`, but then it would be harder to explicitly set the `204` response status this way.

8. Request-body record

```
public record CreatePortfolioRequest(String name) {
}
```

This record is a small data type for the create request. For example, Spring can convert JSON like:

```
{"name":"Retirement"}
```

into a `CreatePortfolioRequest` whose `name` is `"Retirement"`.

How the pieces fit together

For each request, Spring matches the HTTP method and URL to an endpoint, supplies the user and request data, and calls the controller method. The controller passes the work to `PortfolioService`, then returns its result or an appropriate HTTP status.

`ApiExceptionHandler` catches certain exceptions thrown while handling API requests and converts them into consistent HTTP error responses. This keeps error formatting in one place instead of repeating it in each controller.

1. Package and imports

```
package com.portfolio.common;
```

This places the class in the project's `common` package, for code shared across the application.

The imports include:

- `ApiException`, the application's own exception type.
- `ResponseEntity` and `HttpStatus`, used to build HTTP responses.
- `HttpMessageNotReadableException`, which Spring can throw when it cannot read a request body.
- `@ExceptionHandler` and `@RestControllerAdvice`, which connect this class to Spring's request handling.

2. Register a shared exception handler

```
@RestControllerAdvice
public class ApiExceptionHandler {
```

`@RestControllerAdvice` tells Spring that this class can handle exceptions raised by controllers across the application. Its methods return response data directly, typically as JSON.

3. Handle application-specific errors

```
@ExceptionHandler(ApiException.class)
```

```
public ResponseEntity<ApiError> handleApiException(ApiException exception) {
    return ResponseEntity.status(exception.getStatus())
            .body(new ApiError(exception.getCode(), exception.getMessage()));
}
```

`@ExceptionHandler(ApiException.class)` tells Spring to call this method when an `ApiException` is thrown.

The method uses the exception's status, error code, and message to build the response. For example, if the exception contains a `404` status, an error code, and a message, the response uses that status and returns those details in the body.

4. Handle missing or malformed request bodies

```
@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ApiError> handleInvalidBody(HttpMessageNotReadableException
exception) {
```

Spring can throw `HttpMessageNotReadableException` when it cannot convert the request body into the Java type expected by an endpoint—for example, if required JSON is missing or malformed.

This method returns **400 Bad Request** with a generic error response:

```
{
  "error": "invalid_request",
  "message": "Request body is missing or malformed."
}
```

Although the exception is passed into this method, the code doesn't inspect it; it returns the same response for these cases.

5. Define the error-response shape

```
public record ApiError(String error, String message) {
}
```

`ApiError` is a small data type for error responses. Its two fields become JSON properties named `error` and `message`.

Request flow

When a controller or Spring's request-body processing raises one of these exceptions, Spring finds the matching handler method here. The method builds an HTTP response with the appropriate status and JSON error body.

This handler covers the two exception types shown; other exceptions may be handled elsewhere or by Spring's default error handling.

`UserRepository` reads and writes user records in the database. It uses Spring's `JdbcTemplate` to run SQL, and it is called by application code such as `UserAccountService`.

1. Package and imports

```
package com.portfolio.persistence;
```

This places the class in the `persistence` package, which contains code for storing and retrieving application data.

The imports provide:

- `PreparedStatement` and `Statement` for preparing SQL commands.
- `List` and `Optional` for handling query results.
- `UserAccount`, the Java object representing a user.
- `JdbcTemplate`, Spring's helper for running database queries.
- `RowMapper`, which converts a database row into a Java object.
- `GeneratedKeyHolder` and `KeyHolder`, which capture an ID generated by the database.
- `@Repository`, which marks this class as a Spring database component.

2. Convert database rows into `UserAccount` objects

```
private static final RowMapper<UserAccount> USER_MAPPER = (result, row) -> new
UserAccount(
        result.getLong("user_id"),
        result.getString("email"),
        result.getString("password_hash"));
```

When a query returns a row, this mapper reads three columns and creates a `UserAccount`:

- `user_id` becomes the account's ID.
- `email` becomes its email.
- `password_hash` becomes its stored password hash.

`result` represents the current row. `row` is its position in the results, though this mapper doesn't use that position.

3. Store the database helper

```
private final JdbcTemplate jdbc;

public UserRepository(JdbcTemplate jdbc) {
    this.jdbc = jdbc;
}
```

`JdbcTemplate` runs the SQL for this repository. Spring supplies it through the constructor. The `final` field keeps the supplied instance for the lifetime of this repository.

4. Find a user by email

```
public Optional<UserAccount> findByEmail(String email) {
```

This method looks up a user using their email and returns an `Optional<UserAccount>`:

- If a matching user exists, the optional contains the account.
- If no user is found, it is empty.

The query is:

```
SELECT user_id, email, password_hash FROM users WHERE email = ?
```

The `?` is a placeholder. `JdbcTemplate` supplies the `email` value separately, which avoids building the SQL by joining raw text together.

```
return users.stream().findFirst();
```

The query results are stored in a list. This returns the first result, if there is one. The database may enforce email uniqueness; check the table definition to confirm.

5. Create a user

```
public long create(String email, String passwordHash) {
```

This method inserts a user and returns the new database ID.

First, it creates a `KeyHolder` to receive the ID generated by the database. Then `jdbc.update(...)` prepares and runs this SQL:

```
INSERT INTO users (email, password_hash) VALUES (?, ?)
```

The email and password hash are bound to the two placeholders. The code stores a hash rather than the original password.

The `Statement.RETURN_GENERATED_KEYS` option asks JDBC to return the generated ID. After the insert, the method checks for that ID:

```
Number key = keyHolder.getKey();
if (key == null) {
    throw new IllegalStateException("MySQL did not return a user identifier");
}
return key.longValue();
```

If the database did not provide a key, the method throws an exception. Otherwise, it returns the ID as a `long`.

How it fits into the application

The repository handles the database details: SQL queries and mapping rows to objects. Higher-level application code can call `findByEmail` or `create` without needing to write those queries itself.

`LoginThrottleRepository` is a Spring repository that stores and updates login-attempt counters in the `login_throttle` table. It uses Spring's `JdbcTemplate` to run SQL.

1. Repository setup

```
@Repository
public class LoginThrottleRepository {
    private final JdbcTemplate jdbc;
```

`@Repository` marks this class as a persistence component for Spring. Spring supplies a `JdbcTemplate` that handles database connections and query execution.

```
public LoginThrottleRepository(JdbcTemplate jdbc) {
    this.jdbc = jdbc;
}
```

The constructor receives that template and saves it for the methods below.

2. Find or create a throttle bucket: `lockBucket`

```
public ThrottleState lockBucket(String type, byte[] keyHash)
```

A "bucket" is a counter for a particular kind of identifier, such as an account or an IP address. `type` distinguishes the kind; `keyHash` identifies the bucket, apparently in hashed form.

First, the method inserts a bucket with zero failures:

```
INSERT INTO login_throttle (...)
VALUES (?, ?, 0, CURRENT_TIMESTAMP(6))
ON DUPLICATE KEY UPDATE throttle_id = throttle_id
```

If the bucket already exists, the no-op update leaves it unchanged. This relies on a database uniqueness constraint for the bucket key.

Next, it reads the bucket's current state:

```
SELECT failure_count,
       window_started_at <= CURRENT_TIMESTAMP(6) - INTERVAL 15 MINUTE AS
window_expired,
```

```
       delay_until > CURRENT_TIMESTAMP(6) AS delay_active,
       blocked_until > CURRENT_TIMESTAMP(6) AS block_active
...
FOR UPDATE
```

The selected columns include:

- `failure_count`: failures recorded in the current window.
- `window_expired`: whether that window began at least 15 minutes ago.
- `delay_active`: whether a temporary delay is currently in effect.
- `block_active`: whether a temporary block is currently in effect.

`FOR UPDATE` locks the selected database row until the surrounding transaction ends, so concurrent attempts that use this bucket can coordinate their updates. This is effective only when the method runs within an appropriate transaction.

The query maps the database values into a `ThrottleState`:

```
(result, row) -> new ThrottleState(...)
```

If no row is returned, the method throws an exception. Otherwise it returns the first (and expected only) row.

3. Record a failed login: `recordFailure`

```
public void recordFailure(String type, byte[] keyHash, ThrottleState previous)
```

This method calculates the new counter and timing rules from the state returned by `lockBucket`.

```
boolean expired = previous.windowExpired();
int nextCount = expired ? 1 : Math.addExact(previous.failureCount(), 1);
```

If the 15-minute window expired, the new window starts with this failure as count `1`. Otherwise, the count increases by one. `Math.addExact` throws if the integer overflows.

For `ACCOUNT` buckets, it calculates a delay after the fifth failure:

```
int delaySeconds = "ACCOUNT".equals(type) && nextCount >= 5
        ? Math.min(30, 1 << Math.min(5, nextCount - 5))
        : 0;
```

The delay grows exponentially and is capped at 30 seconds. For example, counts 5, 6, 7, 8 produce delays of 1, 2, 4, and 8 seconds; it reaches 30 seconds at higher counts.

For `IP` buckets, it enables a block at 60 failures:

```
boolean block = "IP".equals(type) && nextCount >= 60;
```

The update writes the count and adjusts the relevant timestamps:

```
SET failure_count = ?,
```

```
    window_started_at = IF(?, CURRENT_TIMESTAMP(6), window_started_at),
    delay_until = IF(? > 0, CURRENT_TIMESTAMP(6) + INTERVAL ? SECOND, NULL),
    blocked_until = IF(?, CURRENT_TIMESTAMP(6) + INTERVAL 15 MINUTE, NULL)
```

So an expired window gets a fresh start time; an account delay gets a new deadline (or is cleared when no delay applies); and an IP block gets a 15-minute deadline (or is cleared when no block applies).

4. Reset an account bucket: `resetAccount`

```
public void resetAccount(String type, byte[] keyHash)
```

This sets the failure count to zero, starts a fresh window, and clears any delay:

```
SET failure_count = 0,
    window_started_at = CURRENT_TIMESTAMP(6),
    delay_until = NULL
```

It does **not** clear `blocked_until`. Despite the method name, it accepts any `type`; callers are responsible for passing the intended bucket type.

5. State record

```
public record ThrottleState(
    int failureCount,
    boolean windowExpired,
    boolean delayActive,
    boolean blockActive
) {}
```

This compact record carries the values read by `lockBucket` into the code that decides whether and how to update the bucket.

In short: `lockBucket` creates or locks a bucket and reads its state; `recordFailure` increments it and sets any delay or block; `resetAccount` clears account failure state.

`PortfolioRepository` is the database access layer for portfolios. It uses Spring's `JdbcTemplate` to run SQL and maps database rows into the application's `Portfolio` object.

1. Map database rows to `Portfolio`

```
private static final RowMapper<Portfolio> PORTFOLIO_MAPPER = ...
```

This mapper converts each SQL result row into a `Portfolio`:

- `portfolio_id` becomes the portfolio ID.
- `name` and `currency` become strings.
- `created_at` and `updated_at` are read as SQL timestamps, then converted to Java `Instant` values.

The mapper is reused by both query methods.

2. Store the database helper

```
private final JdbcTemplate jdbc;

public PortfolioRepository(JdbcTemplate jdbc) {
    this.jdbc = jdbc;
}
```

Spring provides the `JdbcTemplate` through the constructor. The repository keeps it to run its queries.

3. Create a portfolio

```
public long create(long userId, String name)
```

This inserts a portfolio owned by `userId`, with the supplied `name` and a default currency of `INR`:

```
INSERT INTO portfolios (user_id, name, currency) VALUES (?, ?, 'INR')
```

The `?` placeholders are filled with `userId` and `name`. A `GeneratedKeyHolder` captures the database-generated portfolio ID. If MySQL doesn't return an ID, the method throws `IllegalStateException`; otherwise, it returns the ID as a `long`.

4. Find all portfolios for an owner

```
public List<Portfolio> findAllByOwner(long userId)
```

This selects the portfolio fields for one user, ordered by creation time and then portfolio ID:

```
WHERE user_id = ?
ORDER BY created_at, portfolio_id
```

Each row is converted using `PORTFOLIO_MAPPER`. The result is a list, which may be empty if the user has no portfolios.

5. Find one portfolio owned by a user

```
public Optional<Portfolio> findByIdAndOwner(long portfolioId, long userId)
```

This selects a portfolio only when both its ID and owner match. It maps the result rows, then returns the first one as an `Optional`:

- A matching portfolio produces `Optional.of(portfolio)`.
- No match produces `Optional.empty()`.

Checking the owner in the query helps callers avoid retrieving another user's portfolio by ID alone.

6. Delete a portfolio owned by a user

```
public int deleteByIdAndOwner(long portfolioId, long userId)
```

This deletes only the row matching both the portfolio ID and owner ID. It returns the number of rows deleted—typically `1` if a portfolio was removed, or `0` if no matching row existed.

7. Check whether a portfolio has holdings

```
public boolean hasHoldings(long portfolioId)
```

The query checks whether at least one row exists in `holdings` for the given portfolio:

```
SELECT EXISTS (
    SELECT 1 FROM holdings WHERE portfolio_id = ?
)
```

The result is converted to a Java `Boolean`, then `Boolean.TRUE.equals(exists)` safely returns `true` only if the query result is `true`.

Overall flow: the repository creates portfolios, retrieves them by owner, deletes an owner's portfolio, and checks whether a portfolio contains holdings.

`HoldingRepository` is the database access layer for holdings. It uses Spring's `JdbcTemplate` to run SQL and turns database rows into `Holding` objects.

1. Map a database row to a `Holding`

```
private static final RowMapper<Holding> HOLDING_MAPPER = ...
```

For each row, the mapper reads:

- `units_owned` into `units`
- `average_purchase_price` into `price`

It then calculates the holding's value:

```
units.multiply(price).setScale(2, RoundingMode.HALF_UP)
```

That multiplies the units by the average purchase price and rounds the result to two decimal places using the usual "half up" rule. The mapper passes the ID, portfolio ID, investment name, ticker, units, price, calculated value, and timestamps to the `Holding` constructor.

2. Keep the database helper

```
private final JdbcTemplate jdbc;

public HoldingRepository(JdbcTemplate jdbc) {
    this.jdbc = jdbc;
}
```

Spring supplies a `JdbcTemplate` to the constructor. The repository uses it for all database operations.

3. Create a holding

`public long create(...)`

The method inserts a holding with its portfolio, display name, normalized name, ticker, units, and average purchase price.

The database generates the new holding ID. `GeneratedKeyHolder` captures it, and the method returns it as a `long`. If the database does not return an ID, the method throws an `IllegalStateException`.

`normalizedName` is stored separately from `name`, likely so application code can compare or search names consistently. Its exact normalization rules are determined by the caller.

4. List holdings in a portfolio owned by a user

`public List<Holding> findAllByPortfolioAndOwner(long portfolioId, long userId)`

The query joins `holdings` to `portfolios`:

```
JOIN portfolios p ON p.portfolio_id = h.portfolio_id
WHERE h.portfolio_id = ? AND p.user_id = ?
```

This returns holdings in the requested portfolio only when that portfolio belongs to the specified user. Rows are ordered by holding ID, mapped through `HOLDING_MAPPER`, and returned as a list. The list can be empty.

5. Find one holding owned by a user

`public Optional<Holding> findByIdAndOwner(long holdingId, long userId)`

This also joins holdings to portfolios, then filters by holding ID and portfolio owner. It returns the first matching row as an `Optional`, or `Optional.empty()` if there is no match.

6. Update a holding owned by a user

`public int updateByIdAndOwner(...)`

The update changes the holding's name, normalized name, ticker, units, and average purchase price. The join to `portfolios` lets the query verify ownership while updating.

It returns the number of rows updated—usually `1` for a match and `0` when no holding with that ID belongs to the user.

7. Delete a holding owned by a user

`public int deleteByIdAndOwner(long holdingId, long userId)`

This deletes the holding only if its portfolio belongs to the specified user. It returns the number of rows deleted.

Overall: every operation involving an existing holding checks ownership through its parent portfolio. The repository creates holdings, lists and finds them, updates them, and deletes them.

Trace a complete request through the app—for example, how creating a holding flows from the controller, through the service, into `HoldingRepository`, and back as a response.

Here's the full flow for **adding a holding**. The endpoint is:

```
POST /api/portfolios/{portfolioId}/holdings
```

For example:

```
POST /api/portfolios/42/holdings
Content-Type: application/json

{
  "investmentName": "Example Fund",
  "ticker": "EXF",
  "unitsOwned": 12.5,
  "averagePurchasePrice": 80.00
}
```

You must be authenticated to call it. Spring Security requires authentication for this route, and the controller receives the logged-in user as a `UserPrincipal`. For a POST request, the app's CSRF protection also applies.

1. Spring routes the request to the controller

In [HoldingController.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/api/HoldingController.java), this method handles the POST:

```
public ResponseEntity<Holding> add(
        @AuthenticationPrincipal UserPrincipal user,
        @PathVariable long portfolioId,
        @RequestBody HoldingService.HoldingInput request)
```

Spring supplies three values:

- `user`: the authenticated user's principal.
- `portfolioId`: the ID taken from the URL.
- `request`: the JSON body, converted to a `HoldingInput` record.

The controller passes the user ID, portfolio ID, and request body to `HoldingService.add(...)`.

2. The service checks portfolio ownership

In [HoldingService.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/application/HoldingService.java), the `@Transactional add` method starts the operation:

```
portfolios.requireOwned(userId, portfolioId);
```

`PortfolioService.requireOwned` looks up the portfolio by both its ID and the user ID. If it doesn't belong to this user—or doesn't exist—it throws a "portfolio not found" API error. This prevents adding a holding to someone else's portfolio.

3. The service validates and prepares the fields

Next, `validate(input)` checks the request and prepares values for storage:

- The investment name is required, trimmed, and limited to 255 characters.
- The name is normalized to Unicode NFC form. A case-folded normalized version is also created for consistent comparisons.
- The ticker is trimmed; an empty ticker becomes `null`; its maximum length is 32 characters.
- Units and average purchase price are required and checked against supported decimal precision.
- Units must be greater than zero; the price must be zero or greater.

If validation fails, the service throws an `ApiException` with a specific error code and HTTP status, usually `400 Bad Request`.

4. The repository inserts the row

The service passes the validated values to `HoldingRepository.create(...)` in [HoldingRepository.java](C:/Users/Admin/portfolio-management-system/src/main/java/com/portfolio/persistence/HoldingRepository.java).

That method inserts them into the `holdings` table using a prepared statement. The database generates a `holding_id`; a `GeneratedKeyHolder` captures it, and the repository returns it. If no ID is returned, it throws an `IllegalStateException`.

If the insert violates a database uniqueness constraint—for example, if the portfolio already contains an equivalent investment—the database exception is caught by the service and translated to a `409 Conflict` with code `duplicate_investment`.

5. The service reads the new row back

After insertion, the service calls:

```
holdings.findByIdAndOwner(id, userId)
```

The repository joins the holding to its portfolio and checks the owner ID again. It maps the row into a `Holding` record. While mapping, it calculates `costBasis` as:

```
unitsOwned × averagePurchasePrice
```

rounded to two decimal places. The service returns the resulting `Holding`.

6. The controller returns the HTTP response

The controller wraps the `Holding` in a response with status `201 Created`:

```
ResponseEntity.status(HttpStatus.CREATED).body(...)
```

Spring serializes the record as JSON. The response contains the holding and portfolio IDs, investment name, ticker, units, average purchase price, calculated cost basis, and creation/update timestamps.

If the service throws an `ApiException`, `ApiExceptionHandler` converts it into a JSON error response containing an `error` code and `message`, with the corresponding HTTP status.

Flow at a glance: HTTP request → `HoldingController.add` → `HoldingService.add` (ownership check and validation) → `HoldingRepository.create` → repository reads the new holding back → controller returns `201 Created` with JSON.